

```
In [8]: import numpy as np
```

```
In [9]: def five_point_first_derivative(f, x, h):
        """
        Five-point central-difference approximation of f'(x)

        f: f(x)
        x: point at f'(x)
        h: step size
        """
        return (f(x - 2*h) - 8*f(x - h) + 8*f(x + h) - f(x + 2*h)) / (12*h)
```

```
In [10]: def five_point_second_derivative(f, x, h):
        """
        Five-point central-difference approximation of f''(x)

        f: f(x)
        x: point at f''(x)
        h: step size
        """
        return (-f(x - 2*h) + 16*f(x - h) - 30*f(x) + 16*f(x + h) - f(x +
        2*h)) / (12*h**2)
```

```

In [11]: # test with sin function
f = np.sin
x0 = 1.0
exact1 = np.cos(x0)
exact2 = -np.sin(x0)

# header row for the results table
print(f"{'h':>10}      {'first-deriv err':>16}      {'ratio':>8}
{'second-deriv err':>18}      {'ratio':>8}")

err01 = err02 = None
for h in (0.1, 0.05, 0.025, 0.0125, 0.00625):
    # five-point approximations
    d1 = five_point_first_derivative(f, x0, h)
    d2 = five_point_second_derivative(f, x0, h)

    # absolute error against the known exact derivatives
    err1 = abs(d1 - exact1)
    err2 = abs(d2 - exact2)

    # ratio of the previous error to this one
    if err01:
        r1 = err01 / err1
    else:
        r1 = float('nan')

    if err02:
        r2 = err02 / err2
    else:
        r2 = float('nan')

    # print results
    print(f"{'h':10.5f}      {'err1':16.3e}      {'r1':8.2f}      {'err2':18.3e}
{'r2':8.2f}")
    err01, err02 = err1, err2

```

h	first-deriv err	ratio	second-deriv err
0.10000	1.799e-06	nan	9.341e-07
0.05000	1.125e-07	15.99	5.842e-08
0.02500	7.035e-09	16.00	3.652e-09
0.01250	4.397e-10	16.00	2.281e-10
0.00625	2.748e-11	16.00	2.047e-11

In []:

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).